

Music Scripting Engine Maintenance Manual

Accessing the Codebase	3
Engine Layer Responsibilities	4
Core Layer (relative filepath: music_engine/core/)	4
Algorithm Layer (relative filepath: music_engine/algorithms/)	4
Engine Layer (relative filepath: music_engine/engine/)	4
Output Layer (relative filepath: music_engine/output/)	4
Setup and Installation of the Codebase	5
Required Dependencies	6
Optional dependencies	6
Running the Engine	6
Method 1: Using the Decorator API (Recommended for Users)	6
Method 2: Using the Direct API (Advanced)	7
Method 3: Using Example Scripts	8
Testing	8
Run All Tests	8
Run Specific Test File	8
Run Specific Test	8
Run with Coverage Report	8
Test File Overview	8
Writing New Tests	9
Common Maintenance Tasks	9
Task 1: Adding a New Algorithm	9
Task 2: Extending Note Pitch Support	10
Task 3: Modifying MIDI Export Parameters	10
Task 4: Improving Playback Quality	10
Task 5: Bug Fixes in Tests	10
Troubleshooting	11
Issue: ImportError when running scripts	11
Issue: Playback produces no sound	11
Issue: MIDI file is empty or corrupted	11
Issue: Notes sound wrong (wrong pitch/duration)	11
Issue: Random walk produces monotonous melodies	12
Development Guidelines	12
Code Style	12

This is a maintenance manual for the upkeep and understanding of the Python music scripting engine. For further information on end-user usage, please refer to the user manual and relevant examples within the codebase.

Accessing the Codebase

The codebase is fully downloadable as a zip file from the main GitHub repository. This can be found at the following link:

<https://github.com/frasermccarter/CS4525-Joint-Honours-Project>

Engine Layer Responsibilities

This section is to provide a surface-level understanding of the engine's architecture and each layer's responsibilities.

Core Layer (relative filepath: `music_engine/core/`)

The core layer is responsible for the lowest level of abstraction and deals with the core mapping of musical concepts to data sets.

`note.py`: Represents individual musical notes with pitch, velocity, and duration.

`sequence.py`: An ordered collection of notes to which algorithmic generation can be applied.

`track.py`: A container for sequences with playback and export capabilities.

`chord.py`: A helper function for creating chord structures (major, minor, seventh, etc.) that are generated using pitch interval sets.

Algorithm Layer (relative filepath: `music_engine/algorithms/`)

The algorithm layer currently contains three algorithms that can be used within user scripts. These algorithms can be altered and expanded upon by the end-users if they so wish.

`random_walk.py`: Generates pitch sequences constrained to scales or chromatic movement.

`rhythm.py`: Generates rhythm patterns using backtracking to meet beat requirements.

`arpeggio.py`: Generates arpeggio patterns from chord symbols.

Engine Layer (relative filepath: `music_engine/engine/`)

`controller.py`: Acts as the central orchestrator for managing tracks, sequences, playback, and export. It is used to expose only the necessary sections of the code to the end-user.

Output Layer (relative filepath: music_engine/output/)

midi_export.py: MIDI file generation using the mido library. More information on mido can be found at the following address:

<https://mido.readthedocs.io/>

playback.py: Real-time audio playback using sounddevice and numpy. More information on sounddevice and numpy can be found at the following addresses, respectively:

<https://python-sounddevice.readthedocs.io/>

<https://numpy.org/>

Setup and Installation of the Codebase

Once the codebase is downloaded from GitHub, unzip the folder and open the folder as a workspace within your preferred IDE (e.g. VSCode or PyCharm).

The root file structure should look as follows:

```
|— music_engine/          # Main package
| |— __init__.py
| |— algorithms/         # Generative algorithms
| | |— arpeggio.py
| | |— random_walk.py
| | |— rhythm.py
| |— core/              # Core musical objects
| | |— chord.py
| | |— note.py
| | |— sequence.py
| | |— track.py
| | |— __init__.py
| |— engine/            # Audio engine
| | |— controller.py
| | |— __init__.py
| |— output/            # Export & playback
| | |— midi_export.py
| | |— playback.py
| | |— __init__.py
| — tests/              # Test suite
```

```

|   |— test_*.py      # Individual module tests
|   |— __init__.py
|— runner.py          # High-level API interface
|— ex[1-6]_*.py       # Example scripts
|— task[1-3].py       # Assignment tasks
|— MAINTENANCE_MANUAL.md # This file
|— output.mid         # Generated MIDI (if any)

```

Required Dependencies

Before running the codebase, it is important that the correct prerequisites/dependencies are installed properly.

- Python 3.7 or higher must be used.
- pip package manager must be used for installing the relevant dependencies. More information on pip can be found at the following address:

<https://packaging.python.org/en/latest/tutorials/installing-packages/>

Enter the command 'pip install mido numpy sounddevice' into the terminal to install the required packages.

Optional dependencies

For running the built-in module tests pytest can be used. Use the command 'pip install pytest' to install its package. More information on pytest can be found at the following address:

<https://docs.pytest.org/>

Running the Engine

Method 1: Using the Decorator API (Recommended for Users)

Create a Python script using the decorator-based interface:

```

from runner import *

@register_track
def my_composition(piano):
    main_melody = piano.add_sequence("melody")
    main_melody.add_note('C4', 1.0)
    main_melody.add_note('D4', 1.0)
    main_melody.add_note('E4', 1.0)

    main_melody.play(tempo=120)

# Call the function once to register it
my_composition(TrackProxy("Piano"))

# Export to MIDI file
export_midi("my_song.mid", tempo=120)

```

Then run with the code with:

```
python your_script.py
```

The script will automatically play the composition at exit and can optionally export to MIDI.

Method 2: Using the Direct API (Advanced)

```

from music_engine.engine.controller import Controller
from music_engine.core.note import Note

# Create controller
controller = Controller()

# Create track
track = controller.new_track("Piano")

# Add sequence
sequence = track.add_sequence("intro")
sequence.add_note('C4', 1.0, velocity=90)
sequence.add_note('E4', 1.0, velocity=90)

# Play
track.play(tempo=120)

# Export
track.export_midi("output.mid", tempo=120)

```

Method 3: Using Example Scripts

The pre-built examples can be run using the following commands:

```
python ex1_creating_scripts.py
python ex2_adding_notes.py
python ex3_using_algorithms.py
python ex4_polyphonic_playback.py
python ex5_exporting.py
python ex6_advanced_examples.py
```

Testing

Run All Tests

```
pytest music_engine/tests/ -v
```

Run Specific Test File

```
pytest music_engine/tests/test_note.py -v
```

Run Specific Test

```
pytest music_engine/tests/test_note.py::test_note_creation -v
```

Run with Coverage Report

```
pip install pytest-cov
pytest music_engine/tests/ --cov=music_engine --cov-report=html
```

Test File Overview

Test File	Purpose	Key Tests
test_note.py	Note creation and validation	Pitch parsing (MIDI, Hz, name)
test_sequence.py	Sequence operations	add_note(), wait(), length
test_track.py	Track management	add_sequence(), composition

test_controller.py	Engine orchestration	Track creation, playback
test_random_walk.py	Pitch generation	Scale/chromatic walks
test_rhythm.py	Rhythm generation	Duration generation
test_arpeggio.py	Chord arpeggio	Arpeggio generation
test_export.py	MIDI file export	File creation validation
test_polyphonic_playback.py	Multi-track playback	Simultaneous playback

Writing New Tests

1. Create test file following naming convention: 'test_*.py'
2. Use 'pytest' fixtures and assertions
3. Place in 'music_engine/tests/'
4. Follow existing test patterns

Example:

```
import pytest
from music_engine.core.note import Note

def test_note_creation_with_name():
    note = Note('C4', 1.0, velocity=80)
    assert note.pitch == 60 # MIDI number
    assert note.duration == 1.0
    assert note.velocity == 80
```

Common Maintenance Tasks

Task 1: Adding a New Algorithm

1. Create a new file in 'music_engine/algorithms/'
2. Implement a generation function returning note data
3. Add imports to existing modules if integrating with Sequence
4. Write tests in 'music_engine/tests/test_[algorithm].py'
5. Document in this manual

Example:

```
# music_engine/algorithms/new_algorithm.py
def generate_pattern(param1, param2):
    """Generate musical pattern."""
    result = []
    # Implementation
    return result
```

Task 2: Extending Note Pitch Support

Current formats: MIDI, Hz, note names, to add a new format (e.g., scale degrees):

1. Edit 'music_engine/core/note.py' - 'Note.__init__()'
2. Add format detection logic
3. Convert to MIDI number
4. Add test case to 'test_note.py'

Task 3: Modifying MIDI Export Parameters

MIDI export controlled in 'music_engine/output/midi_export.py':

Tempo: Passed as parameter to 'export_midi(track, tempo=120)'

Ticks per beat: Default 480, adjustable in function call

Track name: Set via 'mido.MidiTrack.name'

MIDI channel: Set in 'msg = mido.Message('program_change', ...)'

Task 4: Improving Playback Quality

Audio playback in 'music_engine/output/playback.py':

Sample rate: Default 44100 Hz (CD quality)

Waveform: Currently sine wave, can modify 'generate_sine_wave()'

ADSR envelope: Add amplitude enveloping for realistic decay

Polyphonic mixing: Uses simple addition, can implement mixing algorithms

Task 5: Bug Fixes in Tests

1. Run failing test: 'pytest music_engine/tests/test_*.py -v'
2. Identify failure message
3. Navigate to the source module
4. Apply a fix
5. Re-run test: 'pytest music_engine/tests/test_*.py::test_name -v'
6. Verify fix doesn't break other tests: 'pytest music_engine/tests/ -v'

Troubleshooting

Issue: ImportError when running scripts

Symptom: 'ModuleNotFoundError: No module named 'music_engine''

Solution:

1. Ensure you're in the project root directory
2. Add workspace to Python path:

```
import sys
sys.path.insert(0, '/path/to/project/root')
```

3. Or use: 'PYTHONPATH=. python script.py'

Issue: Playback produces no sound

Symptom: 'play()' runs but no audio output

Troubleshooting:

1. Check system volume
2. Verify 'sounddevice' is installed: 'pip install sounddevice'
3. List available audio devices:

```
import sounddevice as sd
print(sd.query_devices())
```

4. Try setting the output device in 'playback.py':

```
sd.default.device = device_id
```

Issue: MIDI file is empty or corrupted

Symptom: Exported MIDI file won't open in DAW

Troubleshooting:

1. Verify track contains sequences: 'print(track.get_sequences())'
2. Verify sequences contain notes: 'print(sequence.get_notes())'
3. Check file was actually written: 'ls -la output.mid'
4. Verify file size > 0 bytes
5. Try exporting with a different tempo: 'export_midi("test.mid", tempo=120)'

Issue: Notes sound wrong (wrong pitch/duration)

Symptom: Played/exported notes don't match input

Debugging:

1. Print note pitch: `'note.get_midi_pitch()'`, `'note.get_frequency()'`
2. Print duration: `'note.get_duration()'`
3. Check velocity scale (0-100 user, 0-127 MIDI internally)
4. Verify tempo affects duration, not note length

Issue: Random walk produces monotonous melodies

Enhancement:

1. Adjust scale: Try 'minor' for a different flavour
2. Increase note count: More notes = more variety
3. Change direction: 'random' vs 'up' / 'down'
4. Combine multiple algorithms in the same sequence

Development Guidelines

Code Style

- Language: Python 3.7+
- Naming: 'snake_case' for functions/variables, 'PascalCase' for classes
- Documentation: Docstrings for all public functions and classes
- Type Hints: Recommended but not enforced

****For questions or updates to this manual, please maintain consistency with the codebase structure and update this document when making significant architectural changes.****